

Chapter 1 - Computer systems

• What is a computer?

Input → Process → Output

A device is considered as a computer if it has both of the following characteristics

• Data-dependent instructions sequence: include conditional branch constructors, whose execution depends on values that can only be calculated during the program execution.

• Data-dependent data selection: must be able to use data to determine the actual data to be used in calculations (Arrays).

The process, is the part that turns the input into outputs. To do this, computers follow a set of instructions known as a program.

These primitive instructions constitute a language, called a machine language.

Computer systems can be designed in an organized, structured, and layered way. It can be run as a set of layers or levels:

(6) Level 6 - user

The user level, it contains software applications;

Level 0 - Digital 'logic' level

It is the machine's true hardware. Its circuits carry out the machine-language programs of level 1.

+ Gate = combination of transistors (tiny electronic component that acts as a miniature switch or an amplifier for electrical signals). Each gate has one or more digital inputs (signals representing 0 or 1) and produces as output some simple function of these inputs, such as AND or OR.

Small number of gates → 1-bit memory combined in groups → register.

Level 1 - Microarchitecture level

Collection of 8 to 32 registers that form a local memory and a circuit called ALU (Arithmetic Logic Unit), which is capable of performing simple arithmetic operations. The interaction between the ALU and registers forms a data path.

On some machines the operation of the data path is controlled by a program called a microprogram. In this case, the microprogram is an interpreter for the instructions at level 2 - it fetches, examines, and executes instructions one by one using the data path to do so. which in other cases, the data path is controlled directly by hardware, working similarly to the last one, but without an explicit stored program to control the interpretation of level 2 instructions.

Level 2 - Instruction Set Architecture level (ISA level)

Every computer is accompanied by an instruction set manual that describes a strictly numerical language, the so-called machine language, they are in fact describing the instructions carried out interpretively by the microprogram or hardware execution circuits.

Level 3 - Operating System machine level

It comes with a set of new instructions, a different memory organization, the ability to run two or more programs concurrently... The new facilities added at level 3 are carried out by an interpreter running at level 2, which has been called an operating system. These level 3 instructions that are identical to level 2's are executed directly by the microprogram, not by the operating system. And that's why this level can be run as an "hybrid" level.

Level 4 - Assembly language

Provides a method for people to write programs for levels 1/2/3 in a not unpleasant form. Programs in assembly language are first translated to level 1/2/3 language by assembler.

Level 5 - Problem-oriented language

Programs written in high-level languages are generally translated to level 3 or level 4 by translators known as compilers.

Hardware ≠ Software

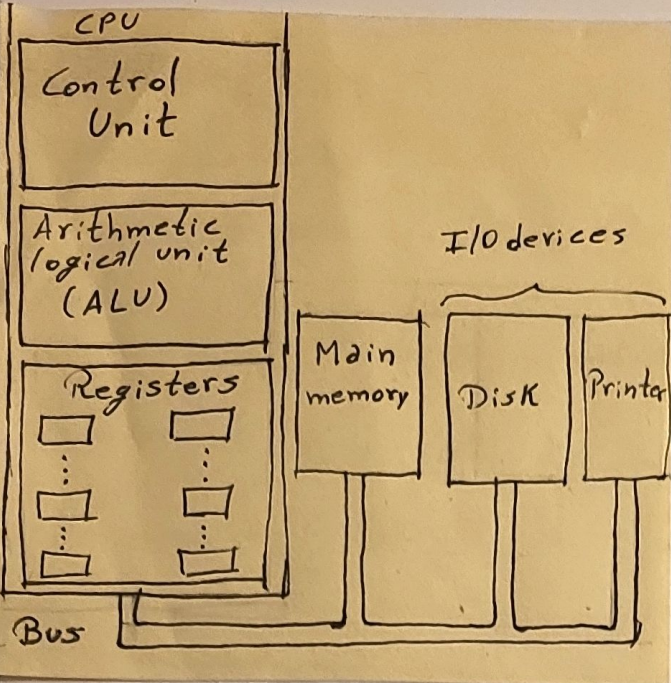
↳ Hardware = consists of tangible objects;

↳ Software = consists of algorithms (detailed instructions telling how to do something) and their computer representations.

- Organization of a computer

Almost all uniprocessor (or scalar) computers follow the so-called von Neumann architecture.

The von Neumann architecture has been extended in several ways to address some of its limitations. A well-known proposal, is the inclusion of a system bus.



The CPU (Central Processing Unit) is the "brain" of the computer. Its function is to execute programs stored in the main memory by fetching their instructions, examining them, and then executing them one after another.

The components are connected by a bus, which is a collection of parallel wires for transmitting address, data, and control signals.

* CPU composition

↳ Control unit is responsible for fetching instructions from main memory and determining their type.

↳ ALU performs operations such as addition and Boolean AND needed to carry out the instructions.

↳ high-speed memory used to store temporary results and certain control information. This memory is made up of a n° of registers.

+ Program Counter (PC) =

points to the next instruction to be fetched for execution

+ Instruction Register (IR) = holds the instruction currently being executed;

↳ CPU Organization

The internal organization of part of a simple von Neumann CPU is called the data path and consists of the registers, the ALU, and several buses connecting the pieces.

The registers feed into two ALU input registers, labeled A and B in the figure. These registers hold the ALU input while the ALU is performing some computation.

The ALU itself performs addition, subtraction, and other simple operations, thus yielding a result in the output register.

* Instructions can be divided into one of two categories:

(1) Register-memory

Allow memory words to be fetched into registers or allow registers to be stored back into memory;

(2) Register-register

A typical instruction fetches two operands from registers, brings them to the ALU input registers, performs some operation on them and stores the result back in one of the registers.

+ Data path cycle = the process of running two operands through the ALU and storing the result;

↳ Instruction Execution

From a conceptual point of view, the execution of an instruction by the CPU can be divided into the following steps:

1. Fetch the next instruction from memory into the instruction register;
2. Change the program counter (PC) to point to the following instruction;
3. Determine the type of instruction just fetched;
4. If the instruction uses a word in memory, determine where it is;
5. Fetch the word in memory, if needed, into a CPU register;
6. Execute the instruction;
7. Go to step (1) to begin executing the following instruction;

The process, called the fetch-decode-execute cycle, is repeated indefinitely, from the time that power is applied to the computer until it is powered off.

Most computer systems have a clock signal that emits periodically a pulse, i.e., it transmits a precise pulse with regular intervals of high and low values. And the time between two consecutive ticks is called a clock cycle (time unit).

Therefore, the speed at which the instructions are executed by the processor depends on the frequency of the clock, measured in cycles per second or Hertz (Hz).

↳ Instruction Level Parallelism

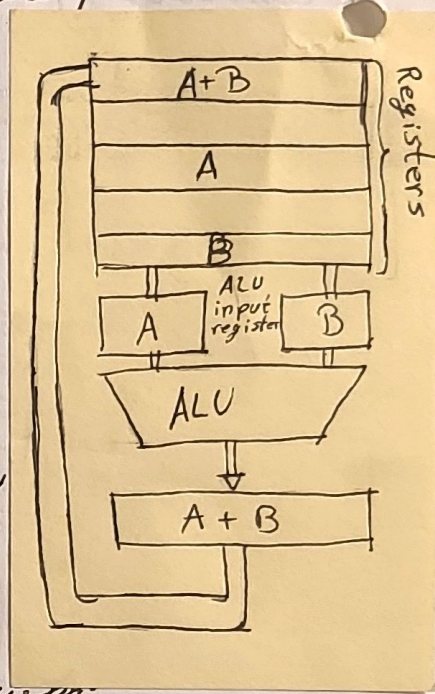
With pipelining, the process of executing instructions is divided in stages, allowing multiple instructions to be overlapped in execution. Each stage is responsible for a part of the process and all can run in parallel.

In fact, instructions throughput, which measures the number of instructions completed per time unit, is improved. This improvement occurs without the need to reduce the time that is required to complete each individual instruction, called the latency.

Theoretically, if all stages take the same time, in a pipeline that contains n stages, then there is a maximum speedup of n times in throughput, even though one individual instruction still has a latency equal to n stages (or n clock cycles).

In fact, the maximum speedup is equal to the number of stages.

The following three major aspects negatively affect the theoretical speed up:



- 1. Resource conflict**
This type of conflict occurs whenever two pipeline stages need to simultaneously access the same resource. This conflict is usually resolved by permitting the operand fetch to occur, while waiting for the instruction fetch.
- 2. Data dependency**
Occurs whenever the result of one instruction, not yet available, is also the operand of a subsequent instruction. Solution is to add delays into the pipeline.
- 3. Branch instructions**
These instructions modify the normal execution flow of a program.

↳ Main Memory
Is the component of the computer, where programs and data are both stored. A memory consists of a number of cells or locations, each of which can store a piece of information. Each cell has a number called its address by which programs can refer to it. A memory with n cells uses addresses from 0 to $n-1$. All memory cells have the same size. In a random access memory (RAM), which stores a piece of data at a unique address and can recall the data upon presentation of the complete unique address. RAM memories are volatile, which means that the information disappears if the supply voltage is turned off.

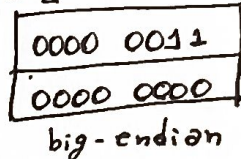
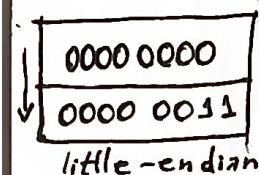
Physically, the main memory consists of a collection of dynamic random access memory (DRAM) chips.

The cell is the smallest addressable unit. (cell = 8-bit / 1 byte);

+ Byte ordering

The bytes in a word can be numbered from right to left, the so called **Little Endian system**, where the bytes are stored from the least significant byte up to the most significant byte, or left to right, system named as **Big Endian**.

Ex: $11\ 0000\ 0000_2$



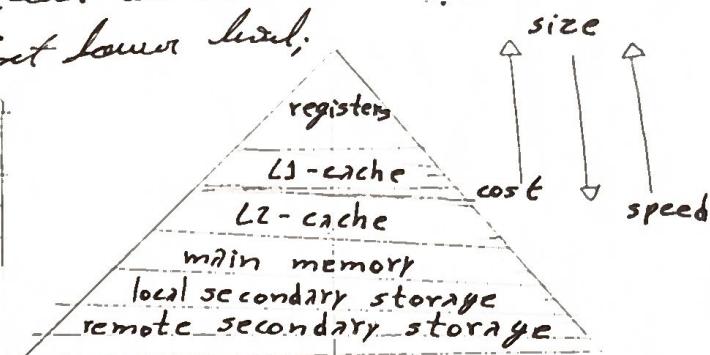
+ Self-modifying code = a concept that is particularly straightforward to perform at the machine-level, by directly writing new instructions over the existing ones in the memory.

↳ Secondary Memory

• Memory hierarchies

The traditional solution to storing a great deal of data is a memory hierarchy. The overall goal of it is to establish a memory system with cost almost as low as the cheapest level and speed almost as fast as the fastest level. All the data in a given level can be found in the next lower level.

level	1	2	3	4
name	registers	cache	main memory	disk storage
typical size	< 1KB	< 16MB	< 512MB	> 1TB
implementation	CMOS	SRAM	DRAM	magnetic disk
access time (ns)	0.25-0.5	0.5-25	50-250	5,000.000
bandwidth (MB/s)	50,000-500K	5K-20K	2500-30K	50-500



↳ I/O devices

An I/O device is a hardware system used by humans or systems to communicate with a computer. So, for each device in a system, there is a device driver, which consists of software that the CPU executes when it has to deal with that device. When a given device produces data that needs to be processed, somehow the CPU needs to take some action. A simple solution, which is not very satisfactory, is for the CPU to make keep checking for incoming data over and over. Whenever it detects that data was made available by the device, it processes. This method is designated as **polling**. To avoid this inefficiency, interrupts are generally used instead of polling. An interrupt is a signal sent by a device to the CPU to request its attention. The CPU reacts by suspending what it is currently doing to answer to the interrupt.

Chapter 2 - Representation of information

The digital abstraction allows that a small variation in a input value is still interpreted correctly, i.e. circuits can be modelled as if they really process 0s and 1s.

- Bits, byte and words.

Bit = a logic value, 0 or 1; $1 \text{ Kibibyte} = 4 \text{ Kibibits}$

$n \text{ bits} = 2^n \text{ entities}$

word = 4 bits, is the basic unit of data handled by a given family of computers.

- Textual information

ASCII (American Standard Code for Information Interchange) - character has 7 bits

Unicode - character has 16 bits.

- Machine - Level instructions

Opcode | arg1 | arg2 | arg3 |

source destination

opcode | rs | rt | rd | shamt | funct |

- Image

Raster image = it's seen as an array of points; Each point is called a pixel; The size of the image needs to be stored, and is part of the metadata (data about data); Colour depth = number of bits used to store each pixel.

Image resolution = is related to how close the pixels are. It is usually measured in dots per inch (dpi).

Vector image

- Audio

Firstly, sound is recorded using a device, like microphone, that translates sound waves into an electrical signal, ~~as it is~~. This process, that reduces a continuous-time signal to a discrete-time signal is called sampling. The value at a point in time is digitized sample.

The samples are then simply converted into binary, using unique binary code. Afterwards, the digital sound can be processed by a computer as a sequence of bits.

Sample resolution is the number of bits used to represent the value of each sample.

Sampling rate number of samples taken per second, measured in Hertz (Hz).

$$\text{size in bits} = f \times r \times t$$

size in bits (bits) sampling rate (Hz) sample resolution (no. of bits) / bytes

Chapter 3 - Representation of numbers

Positional numeral system = represents any number by a sequence of juxtaposed digits.

Radix = base for exponentiation, it is frequently convenient to use radices 10, 8, 16 and 2.

Positional numeral system can use any integer $r \geq 2$ for base, and the digit in position i has weight r^i .

The natural binary numeral system has a fixed range. A representation with n bits ranges from 0 to $2^n - 1$.

It can also be extended to include fractions in any base system, which can be approximated using negative powers of a radix.

(MSD) most-significant digit = the leftmost digit

(LSD) least-significant digit = the rightmost digit

- Octal and hexadecimal

Octal - to - binary & hexadecimal - to - binary

hexadecimal	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
binary	0001	0010	0011	0100	0101	0110	0111	1000	1001	1010	1011	1100	1101	1110	1111
octal	1	2	3	4	5	6	7	10	11	12	13	14	15	16	17

- Negative Binary numbers

Signed magnitude = the leftmost bit is the sign bit (0 is + and 1 is -) and the remaining bits hold the absolute magnitude of the number. $[-(2^{n-1}-1), (2^{n-1}-1)]$

One's complement = has a sign bit with (0 for plus and 1 for minus). To negate a number, replace each 1 by 0 and each 0 by a 1. $[-(2^{n-1}), (2^{n-1}-1)]$

Two's complement has a sign bit, and negating a number is a two-step process. First, apply one's complement. Second, 1 is added to the result. $[-(2^{n-1}), (2^{n-1}-1)]$

- Excess representation

Instead of using a sign bit, excess representation chooses a fixed positive integer

called bias (or offset). $[-b; (2^n - b - 1)]$

To represent any integer V , you add the bias to it to get the unsigned binary value B :

$$\text{Encoded Value (B)} = \text{True Value (V)} + \text{Bias}$$

* to read the value back, you simply subtract the bias.

For an n -bit system, the standard bias used is typically: 2^{n-1} or $2^{n-1} - 1$

- Binary Arithmetic

Addend	0	0	1	1
Augend	+0	+1	+0	+1
Sum	0	1	1	0
Carry	0	0	0	1

Decimal

$$\begin{array}{r} 10 \\ + (-3) \\ \hline +7 \end{array}$$

end-around carry

1's complement

$$\begin{array}{r} 00001010 \\ 11111100 \\ \hline (1)00000110 \\ \hline > 1 \\ \hline 00000111 \end{array}$$

2's complement

$$\begin{array}{r} 00001010 \\ 11111101 \\ \hline (1)00000111 \\ \hline \text{discarded} \end{array}$$

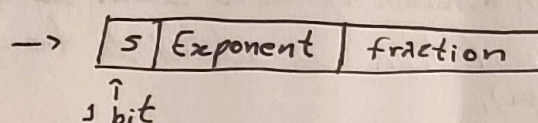
↳ Overflow

In case of unsigned the carry indicates the overflow.

But when it comes to signed numbers we have to be careful when adding two positive numbers, if we get on negative one as a result there was an overflow. Or a case when adding two negative numbers we get a positive one.

- Floating-point numbers

Representation: $n = \text{fraction} \times \text{radix}^{(\text{exponent})}$



↳ Normalization

1. Explicit Normalization: Move the radix point to the left side of the most significant '1' in the bit sequence.

2. Implicit Normalization: Move the radix point to the right side of the most significant '1' in the bit sequence.

We will only use this one, and that's why our $M = f + 1$.

↳ Biasing

We use the excess representation for the exponent, where the bias is equal to $2^{K-1} - 1$, with K being the number of bits dedicated for exponent.

Normal number

$$V = (-1)^s \cdot M \cdot 2^{e - \text{bias}} = (-1)^s \cdot (1 + f) \cdot 2^{e - \text{bias}}$$

bias = $2^{K-1} - 1$
K is the number of bits

Subnormal number

$$V = (-1)^s \cdot M \cdot 2^{1 - \text{bias}} = (-1)^s \cdot (0 + f) \cdot 2^{1 - \text{bias}}$$

+ Every denormalized number is smaller in absolute value than every normalized number

	s	e	M
Normalized	+/-	$0 < e < 2^{K-1} - 1$	$M \geq 1$
Denormalized	+/-	0	$M \neq 0$
Zero	+/-	0	$M = 0$
Infinity	+/-	111...11	$M = 0$
Not a Number	+/-	111...11	$M \neq 0$

1) Métodos de conversão de Floating Point to decimal (Integer Mantissa Method)

2) Decimal to Floating Point Subnormal (ambos no tablet)

Chapter 4.

Stack and Procedure

• What is a procedure?

Also called function/method, is a block of code that:

- Has a well-defined set of arguments and a return value;
- Can be called from different points in a program;
- Hides implementation while exposing a clear interface;

• Stack

Is a fundamental data structure used in computer programming, particularly in assembly language. It operates on a Last-In, First-Out (LIFO) principle.

• Push: Adding data to the top of the stack;

• Pop: Removing data from the top of the stack;

• Stack Pointer (SP): A CPU register that always points to the top of the stack;

• Base Pointer (BP): A CPU register that always points to the "floor" of the stack

- Stack Frame

When a function/procedure is called, a dedicated section of the stack is created for it, known as a stack frame or activation record.

A typical stack frame contains:

↳ Arguments: Values passed from the calling function to the current function.

↳ Return Address: The memory address of the instruction in the calling function to which execution should return after the current function finishes.

↳ Base Pointer: Push the ebp to the stack, to save the "floor" of the stack frame of the caller procedure. It is used by the caller procedure as the base pointer to access all the elements in stack frame.

↳ Saved Registers: Values of registers that need to be preserved across the function call (callee-saved registers)

+ Register Conventions

Type	Registers
Callee-saved	ebx, esi, edi, ebp
Caller-saved	eax, ecx, edx

Who saves it?

proc B must preserve these

proc A must preserve these before calling

↳ Local Variables: Memory space allocated for variables declared within the function.

• Key instructions

Instruction	Effect	Description
call l	push eip; eip ← 1	call procedure at label;
call %o	push eip; eip ← 0	call procedure at register/memory;
leave	esp ← ebp; pop ebp	tear down stack frame;
ret	pop eip	return to caller;

• Procedure call and Execution flow

When one procedure (the caller, procA) calls another procedure (the callee, procB), a series of steps occur:

1) Argument Passing: procA provides arguments to procB. This can be done by:

• Pushing arguments onto the stack (often from right to left in C);

• Passing arguments in CPU registers

2) Control Transfer (call instruction)

• The call instruction is used to transfer control to procB.

• Crucially, it automatically pushes the return address onto the stack.

• The Instruction Pointer (IP) is then updated to the starting address of procB;

3) Stack Frame setup

• Save ebp: procB first pushes the current ebp.

• Set new ebp: procB then sets its own ebp.

• Save callee-saved registers: If procB needs to use registers, it pushes their original values;

```
push %ebp
movl %esp, %ebp
```

• Allocate space for local variables: decrements the stack pointer (esp) to reserve space for its local variables. `subl $N, %esp`

4) Execution of proc B

5) Stack Frame Teardown

- ↳ Store return value in eax `movl result, %eax`
- ↳ Deallocate local variables `addl $N, %esp`
- ↳ Restore callee-saved registers `popl %edi...`
- ↳ Restore base pointer and stack p. `leave`

6) Return to caller (ret)

- The ret instruction pops the return address from the top of the stack.
- The SP is set to this return address, transferring control back to proc A.

7) Caller cleanup

- proc A may need to clean up any space it allocated on the stack for arguments.
- proc A reads the return value from eax before restoring any callee-saved registers. Then pops callee-saved registers in reverse order from how they were pushed.

• Accessing Stack Frame Elements via ebp offset

Example:

val	-12
i	-8
ebx	-4
ebx	0
ret address	+4
n	+8
v	+12
...	

Chapter

Chapter 5 - Cache memory

- Cache terms
 - Miss: the memory element was not found at that particular memory level;
 - Hit: element was found at memory level;
 - Hit rate: percentage of time that the element was found at a particular level.

$$r_h = \frac{\text{number of hits}}{\text{total number of access}} \times 100\%$$

- Miss rate:

$$r_m = 1 - \text{hit rate}$$

- Effective Access time: from the point of view of the processor, what's the average time to get the memory element, considering that sometimes it is going to be getting it out of the cache, sometimes it's going to be getting it out of main memory. It can also be called as the mean access time (t_{ma}).
 - cache access time
 - main memory access time

$$t_{ma} = r_h \cdot t_c + r_m \cdot t_m$$

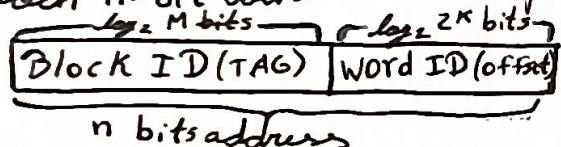
- Principle of locality

- Temporal locality: implies that recently accessed data or instructions are likely to be needed in the near future;
- Spatial locality: states that items whose memory addresses are close to each other tend to be required close together in time;

- Memory-cache system

The main memory is composed of 2^n addressable words, with each one having a unique n -bit address.

Each n -bit address is implicitly split into two parts:

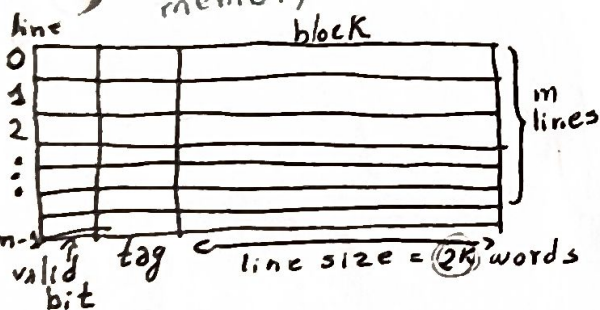


- Block ID (Tag): Identifies which block from main memory is currently stored in a cache line;
- Word ID (Offset): Identifies a specific word (data element) within a block;

- + The number of bits used for the Word ID determines the block size. If there are K bits for the word ID, the block size is 2^K words.

- + Block: a fixed-length group of 2^K words transferred between memory and cache as a unit.

$$\text{number of blocks in memory } M = \frac{2^n}{2^K} \rightarrow \text{total addressable words in memory}$$



- + Cache line: portion of the cache memory that can store one block.
- Valid bit: a control bit indicating whether the data in that line is currently valid. Set to reset valid on system boot;
- Control bits (not shown): bit to indicate whether the line has been modified since it was loaded into the cache.
- Line size: number of data in cache line, equal to block size.

- Mapping Function

- ↳ Fully Associative mapping

Allows any memory block to be loaded into any cache line.

Each n -bit address is divided into only 2 fields:

Field	Name	Size	Purpose
0	word offset	$0 - \log_2 2^K$ bits	Identifies a word within the block;
t	tag	$t = n - 0$	Identifies which memory block is in that line;

- ↳ Direct Mapping

Each memory block maps to exactly one possible cache line - determined by a modular formula.

Which line does a block go to?

$$i = j \bmod m$$

i = cache line number (destination)
 j = memory block number
 m = total number of cache lines.

Direct mapping splits each n -bit address into 3 fields



Field	Name	Size	Purpose
0	word offset	$0 = \log_2 2^k \text{ bits}$	Identifies a word within the block;
s	line index	$s = \log_2 m \text{ bits}$	Identifies which cache line to look in;
t	tag	$t = n - s - 0$	Identifies which of the many blocks mapped to that line is stored there;

Advantages

Simple and cheap - only one tag comparison needed per lookup

Disadvantages

Each block has a unique forced line - if two frequently used blocks map to the same line, they constantly evict each other;

Set-Associative Mapping

Instead of mapping to a single line, each address maps to a set containing N lines:
 N -way set associative = N lines per set $N^\circ \text{ of sets} = \frac{m}{N}$ → lines in cache

Identical structure to direct mapping, but s now indexes a set, not a single line:



Field	Name	Size	Purpose
0	word offset	$0 = \log_2 2^k \text{ bits}$	"
s	set index	$s = \log_2 (n^\circ \text{ of sets}) \text{ bits}$	Identifies which set to look in
t	tag	$t = n - s - 0$	"

Replacement Algorithm

It decides in which cache line to store the new block.

- LRU: least recently used;
- NMRU: next most-recently used;
- FIFO: first in first out;
- LFC: least frequently used;

Chapter 6 - Code Optimisation

Optimisation aims improve attributes, particularly execution, speed, program size, and increasingly, energy consumption.
These criteria conflict - you cannot maximise one without sacrificing another. ~~like~~

- Key concepts

- **Trade-offs**: Optimisation often involves balancing competing non-functional requirements. For instance, optimising for performance might negatively impact maintainability, making the code harder to read and modify.
- **Algorithmic vs Coding level**: Efficient code can be improved at two levels:
 - **Algorithmic**: Selecting more efficient algorithms and data structures.
 - **Coding**: Writing source code that compilers can effectively transform into fast or small machine code.
- **Compiler Capabilities**: Compilers are sophisticated tools that can automatically apply various optimisations.
- **Programmer's Role**: Understanding how compilers optimise and what constitutes an "optimisation blocker" allows programmers to write code that is more amenable to compiler optimisation.

- Limitations of Compilers

- **Preservation of Behaviour**: Optimisations must be "safe", meaning the resulting program behaves identically to the unoptimised version under all possible execution paths.
- **Limited Program View**: Compilers often have a localised view of the program, making it difficult to perform broader, program-wide optimisations.
- **Compilation Time**: The optimisation process itself must be reasonably fast. Significant time spent optimising for marginal gains is not practical.
- **Logical Conditions**: Compilers typically do not optimise for conditions that are logically impossible or highly improbable, even if technically possible within the language's rules.
- **Optimisation Blockers**:
 - **Memory Aliasing**: This occurs when two or more different pointers can refer to the same memory location. A compiler cannot assume that pointers point to distinct locations, which can prevent certain optimisations.

```
int pr1(int *x, int *y) {  
    *x += *y;  
    *x += *y;  
}  
int pr2(int *x, int *y) {  
    *x += 2 * *y;  
}
```

- **Functions Side Effects**: A function has a side effect if it modifies variables outside its local scope or has an observable effect beyond returning a value. Compilers are cautious about optimising calls to functions with potential side effects.

```
int counter = 0;  
int f(int p) { return (p + counter++); }  
int func1(int x) { return (f(x) + f(x)); }  
int func2(int x) { return (2 * f(x)); }
```

$func1(5) = 5 + 0 + 1 + 5 + 2 = 13$, while $func2(5) = 2(5 + 0) = 10$.

- **Hidden loop dependencies**: Compilers often treat functions as "black boxes" they cannot analyse. This prevents optimisations like moving function calls out of loops if the function's behaviour might depend on variables modified within the loop.
for($i=0$; $i < strlen(s)$; $i++$) Compiler keeps $strlen$ inside the loop because it might have side effects or the string might change.

- Types of Optimisations

↳ Machine-Independent Optimisations

These optimisations improve code without considering specific processor architecture details.

- **Code Motion**: moving statements or expressions from inside a loop to outside, provided their semantics are not changed. This reduces redundant computations.
- **Elimination of Unnecessary Memory Access**: replacing memory accesses with register operations, i.e. accumulating into a local variable instead of writing to memory every iteration, as registers are significantly faster.

```
*value = 0;
for (...) {
    *value += a[i];
}
```

```
int acc = 0;
for (...) {
    acc += a[i];
}
*value = acc;
```

Single memory write at the end

- **Loop Unrolling**: reducing loop overhead (incrementing, comparing, jumping) by performing multiple iterations work within a single loop iteration. This increases each size but can significantly speed up execution.

The trade-off is between execution time and code size. More unrolling reduces loop overhead but increases code size.

- **Reduction of Procedure Calls**:

Replacing a function call with the actual body of the function. This eliminates the overhead associated with function calls (stack management, jumps).

$$1 + (3+K) \times \frac{n}{K} + (n \bmod K)$$

Block size
array size

no of instructions executed (IA32)

Machine-Dependent Optimisations

- **Instruction Selection**: Choosing the most efficient machine instruction for a given high-level operation.
- **Replace multiplications**: replacing computationally expensive operations with cheaper ones.

$y = 8 * a;$ $\rightarrow$ $\begin{cases} \text{mov } \%eax, \%ebx // \text{copy 'a' to 'y'} \\ \text{sll } \$3, \%ebx // \text{left shift by 3 (equivalent to multiplying by } 2^3) \end{cases}$

- **Efficient Address Calculation**: Using the processor's addressing modes to calculate memory addresses efficiently.

Cache-Oriented Optimisations

- **Locality of Reference**

- **Temporal Locality**

- **Spatial Locality**

- **Stride**

The stride is the distance in memory between successive accesses to elements of an array.

- **Stride-1**: sequential access = best spatial locality
- **High stride**: jumping around memory = poor spatial locality